

Measurement checks – HPT (Czech data)

DIF & multi-group CFA by ideological attitude (ideological contamination tests)

HPT and Extremism project

2026-10-02

Contents

1	Purpose & plan	1
2	Setup	2
3	Data	2
3.1	Scoring & grouping	3
4	Descriptives (checks)	4
5	DIF analysis (ordinal, item-by-item)	5
6	Multi-group CFA (configural → metric → scalar)	8
7	(Optional) Two-factor robustness check	10
8	Interpretation & reporting	11
8.1	DIF summary	11
8.2	MG-CFA summary	11
9	Reproducibility appendix	12

1 Purpose & plan

We assess whether *ideological attitude groups* respond differently to the HPT items **even at equal underlying HPT competence**. Concretely:

- Create **low/high ideology** groups from FR-LF-mini and KSA-3 (see codebook).
- Run **joint omnibus GRM DIF** tests for each HPT item.

- Run **multi-group CFA** (configural → metric → scalar).
- The registered H4 concerned positive ideology-related DIF on CONT items, evaluated with a continuous-ideology MIMIC analysis. The current joint omnibus GRM DIF and MG-CFA analyses are post-registration and do not implement the registered continuous-ideology MIMIC analysis. Evidence of DIF or scalar non-invariance would indicate measurement noninvariance between the extreme ideology groups, limiting score comparability without itself establishing a directional contamination mechanism.

2 Setup

```
options(width = 120)
library(dplyr)
library(tidyr)
library(ggplot2)
library(psych)
library(knitr)
library(stringr)
library(janitor)
library(difR)      # DIF for ordinal items
library(lavaan)    # CFA / invariance
library(semTools)  # helpers
library(car)       # recode
library(mirt)
source("scoring_helpers.R")
```

3 Data

```
# Load the dataset created in 00_data-preparation
normalised_responses <- readRDS("student_responses.RDS")
stopifnot(exists("normalised_responses"))
dat_raw <- normalised_responses

# Cluster vars
dat_raw <- dat_raw %>%
  mutate(
    school_id = as.factor(school_id),
    class_label = as.factor(class_label),
    class_id = interaction(school_id, class_label, drop = TRUE)
  )

# Coerce HPT items to numeric early (1-4 expected in codebook)
hpt_items <- c(paste0("POP",1:3), paste0("ROA",1:3), paste0("CONT",1:3))
```

```

dat_raw <- dat_raw %>% mutate(across(all_of(hpt_items), ~ suppressWarnings(as.numeric(.))))

# Reverse POP item-wise (so higher = more contextualised)
POP_rev_items <- paste0("POP", 1:3)
dat_raw <- dat_raw %>%
  mutate(across(all_of(POP_rev_items), ~ 5 - ., .names = "{.col}_rev")) %>%
  mutate(
    HPT_POP_raw = scale_mean(., POP_rev_items, min_answered = 2),
    HPT_POP_rev = scale_mean(., paste0(POP_rev_items, "_rev"), min_answered = 2),
    HPT_CONT    = scale_mean(., paste0("CONT", 1:3), min_answered = 2),
    HPT_ROA     = scale_mean(., paste0("ROA", 1:3), min_answered = 2),

    # Canonical composites (CTX6 is our stable default)
    HPT_CTX6 = rowMeans(cbind(HPT_POP_rev, HPT_CONT), na.rm = FALSE),
    HPT_TOT9 = rowMeans(cbind(HPT_POP_rev, HPT_CONT, HPT_ROA),
                        na.rm = FALSE)
  )

```

We use variables as defined in the **codebook** (metadata; KN1-KN6; POP1-POP3; ROA1-ROA3; CONT1-CONT3; FR-LF mini RD1-RD3 & NS1-NS3; KSA-3 A1-A3, U1-U3, K1-K3; SDR1-SDR5).

3.1 Scoring & grouping

- **HPT items** are 1-4. We reverse only *presentism* items (POP1-POP3: 5 - POP*) so that a single higher-is-better direction is used for scale construction and MG-CFA. DIF uses original item codings (reversal is not required for DIF detection).
- **Ideology composite**: FR-LF-mini (RD1-3 + NS1-3) and KSA-3 (9 items). We z-score the two scale means and average them → **IDEO_Z**. *Low* = bottom 33%, *High* = top 33% (middle third excluded to sharpen contrasts).
- **Controls**: prior knowledge (sum KN1-KN6), social desirability (SDR1-SDR5; note SDR2-SDR4 are already reversed upstream per codebook).

```

# Select blocks
frlf_items <- c(paste0("RD",1:3), paste0("NS",1:3))
ksa_items  <- c(paste0("A",1:3), paste0("U",1:3), paste0("K",1:3))
kn_items   <- paste0("KN",1:6)
sdr_items  <- paste0("SDR",1:5)

# Coerce predictors to numeric
num_blocks <- c(frlf_items, ksa_items, kn_items, sdr_items)

dat <- dat_raw %>%
  mutate(across(all_of(num_blocks), ~ suppressWarnings(as.numeric(.)))) %>%
  # Scale scores
  mutate(
    HPT_total = HPT_TOT9,

```

```

HPT_POPR = HPT_POP_rev,
FRLF_mean = scale_mean(., frlf_items, min_answered = 4),
KSA_mean = scale_mean(., ksa_items, min_answered = 7),
KN_sum = rowSums(across(all_of(kn_items)), na.rm = TRUE),
SDR_mean = scale_mean(., sdr_items, min_answered = 4)
) %>%
mutate(
  FRLF_z = as.numeric(scale(FRLF_mean)),
  KSA_z = as.numeric(scale(KSA_mean)),
  IDEO_Z = (FRLF_z + KSA_z) / 2
)

# Define tertile groups
qs <- quantile(dat$IDEO_Z, probs = c(.3334, .6666), na.rm = TRUE)
dat <- dat %>%
  mutate(
    ideology_group = case_when(
      IDEO_Z <= qs[1] ~ "Low",
      IDEO_Z >= qs[2] ~ "High",
      !is.na(IDEO_Z) ~ "Mid"
    )
  )

kable(dat %>% count(ideology_group), caption = "Group sizes (Low/High ideology; Mid excluded from group-wise tests)")

```

Table 1: Group sizes (Low/High ideology; Mid excluded from group-wise tests)

ideology_group	n
High	95
Low	96
Mid	92
NA	10

Note. We focus on *Low* vs *High* to maximise contrast for DIF/MG-CFA. *Mid* is retained for descriptives only.

4 Descriptives (checks)

```

desc_tbl <- dat %>%
  group_by(ideology_group) %>%
  summarise(n = n(),

```

```
HPT_total = mean(HPT_total, na.rm = TRUE),
KN_sum     = mean(KN_sum,     na.rm = TRUE),
SDR_mean   = mean(SDR_mean,  na.rm = TRUE)) %>%
  arrange(match(ideology_group, c("Low", "Mid", "High")))
kable(desc_tbl, digits = 2, caption = "Descriptives by ideology group (means)")
```

Table 2: Descriptives by ideology group (means)

ideology_group	n	HPT_total	KN_sum	SDR_mean
Low	96	2.89	3.33	3.16
Mid	92	2.84	2.89	3.00
High	95	2.76	2.96	2.88
NA	10	2.67	2.30	NaN

5 DIF analysis (ordinal, item-by-item)

Goal. Do Low/High ideology groups differ on individual HPT items in a joint omnibus likelihood-ratio DIF test? Each test jointly releases that item's slope and threshold constraints. Raw p -values are Bonferroni-adjusted across the nine items and evaluated at familywise $\alpha = .05$ (equivalent per-test $\alpha = .0056$).

```
## Keep only Low/High groups
anal <- dat[dat$ideology_group %in% c("Low", "High"), , drop = FALSE]

## HPT item list in original coding
hpt_items <- c("POP1", "POP2", "POP3", "ROA1", "ROA2", "ROA3", "CONT1", "CONT2", "CONT3")
stopifnot(all(hpt_items %in% names(anal)))

## Build item matrix
hpt_mat <- anal[, hpt_items, drop = FALSE]
for (j in seq_along(hpt_items)) hpt_mat[[j]] <- suppressWarnings(as.numeric(hpt_mat[[j]]))

## Group factor
grp <- factor(anal$ideology_group, levels = c("Low", "High"))

## Drop rows with <2 answered items
keep <- rowSums(!is.na(hpt_mat)) >= 2
hpt_mat <- hpt_mat[keep, , drop = FALSE]
grp <- droplevels(grp[keep])

stopifnot(nrow(hpt_mat) == length(grp), nlevels(grp) == 2)
print(table(grp))
```

```

## grp
## Low High
## 96 95
# Constrained multi-group graded model, then DIF with scheme="drop"
mod_base <- multipleGroup(
  data      = hpt_mat,
  model     = 1,
  group     = grp,
  itemtype  = "graded",
  invariance = c("slopes", "intercepts", "free_means", "free_var")
)

params_all <- mirt::mod2values(mod_base)$name
unique_pars <- unique(params_all)
pars_to_test <- c(
  grep("^a", unique_pars, value = TRUE),
  grep("^d\\d+$", unique_pars, value = TRUE)
)
stopifnot(length(pars_to_test) > 0)

dif_out <- DIF(
  mod_base,
  which.par = pars_to_test,
  scheme    = "drop",
  items2test = colnames(hpt_mat),
  p.adjust  = "bonferroni",
  verbose   = FALSE
)

res_tbl <- as.data.frame(dif_out) %>%
  tibble::rownames_to_column("Item") %>%
  transmute(
    Item,
    X2,
    df,
    p,
    adj_p,
    Flag = ifelse(adj_p < .05, "YES", "no")
  )

kable(res_tbl, digits = 3,
  caption = "DIF omnibus item tests (mirt; graded). Each test jointly releases slope and threshold constraints. Raw p-values and Bonferroni

```

Table 3: DIF omnibus item tests (mirt; graded). Each test jointly releases slope and threshold constraints. Raw p-values and Bonferroni-adjusted p-values are shown; familywise alpha = .05 (per-test alpha = .0056).

Item	X2	df	p	adj_p	Flag
POP1	10.464	4	0.033	0.300	no
POP2	9.012	4	0.061	0.547	no
POP3	9.723	4	0.045	0.408	no
ROA1	9.585	4	0.048	0.432	no
ROA2	3.133	4	0.536	1.000	no
ROA3	8.254	4	0.083	0.744	no
CONT1	1.509	4	0.825	1.000	no
CONT2	3.602	4	0.462	1.000	no
CONT3	9.631	4	0.047	0.424	no

```
flagged <- res_tbl$Item[res_tbl$Flag == "YES"]
if (length(flagged) > 0) {
  which_item <- which(colnames(hpt_mat) == flagged[1])
  plot(mod_base, type = "trace", which.items = which_item,
        facet_items = FALSE, groups = levels(grp))
} else {
  plot.new(); text(0.5, 0.5,
                  "No omnibus DIF item was flagged after Bonferroni adjustment (familywise alpha = .05).")
}
```

No omnibus DIF item was flagged after Bonferroni adjustment (familywise alpha = .05).

```
grm_coefficients <- coef(mod_base, IRTpars = FALSE, simplify = TRUE)
stopifnot(isTRUE(all.equal(
  grm_coefficients$Low$items,
  grm_coefficients$High$items,
  check.attributes = FALSE
)))

table_s1_for_group <- function(group) {
  as.data.frame(grm_coefficients[[group]]$items) %>%
    tibble::rownames_to_column("Item") %>%
    transmute(Item, a1, d1, d2, d3)
}

table_s1 <- bind_rows(
  High = table_s1_for_group("High"),
  Low = table_s1_for_group("Low"),
```

```

    .id = "Group"
  ) %>%
    arrange(Item, factor(Group, levels = c("High", "Low"))) %>%
    mutate(across(c(a1, d1, d2, d3), ~ round(.x, 3)))

write.csv(table_s1, "../outputs/table_s1_irt_parameters.csv", row.names = FALSE)
kable(filter(table_s1, Group == "High") %>% select(-Group), digits = 3,
       caption = "Table S1 source: constrained multi-group GRM item parameters from mod_base. Parameters are equal across Low and High ideology groups")

```

Table 4: Table S1 source: constrained multi-group GRM item parameters from mod_base. Parameters are equal across Low and High ideology groups.

Item	a1	d1	d2	d3
CONT1	1.853	2.801	0.733	-1.958
CONT2	1.414	2.098	0.728	-1.311
CONT3	1.359	2.755	0.755	-1.319
POP1	-1.017	0.198	-1.369	-2.671
POP2	-0.351	0.896	-0.860	-2.210
POP3	-0.643	0.702	-1.063	-2.728
ROA1	1.181	2.620	1.192	-0.796
ROA2	0.708	2.148	0.492	-1.770
ROA3	1.416	2.690	1.297	-0.946

6 Multi-group CFA (configural $\rightarrow$ metric $\rightarrow$ scalar)

Model. We specify a **three-factor model** (POP_rev, ROA, CONT) with POP items reversed so that all loadings point to *more HPT-congruent* responses. We then test invariance across Low vs High ideology groups.

```

# Build analysis frame with reversed POP and intact ROA/CONT
cfad <- dat %>%
  filter(ideology_group %in% c("Low", "High")) %>%
  transmute(
    ideology_group, class_id,          # keep cluster id for reference (not used by lavaan here)
    POP1 = 5 - POP1,
    POP2 = 5 - POP2,
    POP3 = 5 - POP3,
    ROA1, ROA2, ROA3,
    CONT1, CONT2, CONT3
  )

# Coerce all item columns to numeric and enforce ordinal 1:4 range; replace out-of-range with NA
ord_items <- setdiff(names(cfad), c("ideology_group", "class_id"))

```



```

cfad <- cfad %>% mutate(across(all_of(ord_items), ~ suppressWarnings(as.numeric(.))))
cfad <- cfad %>% mutate(across(all_of(ord_items), ~ ifelse(. %in% 1:4, ., NA_real_)))

# If any columns had non 1-4 values, warn instead of stopping
bad_cols <- vapply(cfad[ord_items], function(x) any(!is.na(x) & !(x %in% 1:4)), logical(1))
if (any(bad_cols)) {
  warning(sprintf("Non-1:4 values were set to NA in: %s", paste(names(bad_cols)[bad_cols], collapse=", ")))
}

model_3f <- '
  POP =~ POP1 + POP2 + POP3
  ROA =~ ROA1 + ROA2 + ROA3
  CONT =~ CONT1 + CONT2 + CONT3
'

# Run invariance ladder with WLSMV on ordered items; DO NOT pass cluster (not supported with ordered)
fit_conf <- cfa(model_3f, data = cfad, group = "ideology_group",
  ordered = ord_items, estimator = "WLSMV")

fit_metr <- cfa(model_3f, data = cfad, group = "ideology_group",
  ordered = ord_items, estimator = "WLSMV",
  group.equal = c("loadings"))

fit_scal <- cfa(model_3f, data = cfad, group = "ideology_group",
  ordered = ord_items, estimator = "WLSMV",
  group.equal = c("loadings", "thresholds"))

get_fit <- function(fit) {
  fitMeasures(fit, c("chisq.scaled", "df.scaled", "pvalue.scaled",
    "cfi.scaled", "rmsea.scaled", "srmr"))
}

fits <- bind_rows(
  Configural = get_fit(fit_conf),
  Metric      = get_fit(fit_metr),
  Scalar      = get_fit(fit_scal)
) %>%
  mutate(Model = c("Configural", "Metric", "Scalar")) %>%
  select(Model, everything())

fits %>% mutate(across(where(is.numeric), round, 3)) %>%
  kable(caption = "MG-CFA fit indices by invariance level (WLSMV).")

```

Table 5: MG-CFA fit indices by invariance level (WLSMV).

Model	chisq.scaled	df.scaled	pvalue.scaled	cfi.scaled	rmsea.scaled	srmr
Configural	56.436	48	0.189	0.977	0.044	0.074
Metric	62.857	54	0.191	0.976	0.043	0.081
Scalar	69.871	63	0.258	0.981	0.035	0.076

```
deltas <- tibble(
  step = c("Configural -> Metric", "Metric -> Scalar"),
  dCFI = c(fits$cfi.scaled[2] - fits$cfi.scaled[1], fits$cfi.scaled[3] - fits$cfi.scaled[2]),
  dRMSEA = c(fits$rmsea.scaled[2] - fits$rmsea.scaled[1], fits$rmsea.scaled[3] - fits$rmsea.scaled[2])
)

deltas %>% mutate(across(where(is.numeric), round, 3)) %>%
  kable(caption = "Delta fit (CFI, RMSEA) across steps.")
```

Table 6: Delta fit (CFI, RMSEA) across steps.

step	dCFI	dRMSEA
Configural -> Metric	-0.001	-0.001
Metric -> Scalar	0.005	-0.008

How to read this. If **metric holds** (small $\Delta\text{CFI}/\Delta\text{RMSEA}$), loadings are equivalent. If **scalar fails**, thresholds differ and group-mean comparisons are measurement-noninvariant; this warrants cautious interpretation but does not itself establish a directional ideological effect.

7 (Optional) Two-factor robustness check

```
model_2f <- '
  CTX =~ POP1 + POP2 + POP3 + CONT1 + CONT2 + CONT3
  ROA =~ ROA1 + ROA2 + ROA3
'

measurementInvariance(model_2f, data = cfad, group = "ideology_group",
  estimator = "WLSMV",
  ordered = ord_items)
```

8 Interpretation & reporting

8.1 DIF summary

```
kable(res_tbl, caption = "DIF results (for reference in text).")
```

Table 7: DIF results (for reference in text).

Item	X2	df	p	adj_p	Flag
POP1	10.464417	4	0.0332907	0.2996161	no
POP2	9.012104	4	0.0607976	0.5471788	no
POP3	9.723491	4	0.0453521	0.4081691	no
ROA1	9.585251	4	0.0480247	0.4322223	no
ROA2	3.133315	4	0.5357685	1.0000000	no
ROA3	8.254446	4	0.0826898	0.7442083	no
CONT1	1.509152	4	0.8250191	1.0000000	no
CONT2	3.602325	4	0.4624911	1.0000000	no
CONT3	9.631130	4	0.0471214	0.4240930	no

8.2 MG-CFA summary

```
kable(fits %>% mutate(across(where(is.numeric), round, 3)),  
      caption = "MG-CFA fit to reference in text.")
```

Table 8: MG-CFA fit to reference in text.

Model	chisq.scaled	df.scaled	pvalue.scaled	cfi.scaled	rmsea.scaled	srmr
Configural	56.436	48	0.189	0.977	0.044	0.074
Metric	62.857	54	0.191	0.976	0.043	0.081
Scalar	69.871	63	0.258	0.981	0.035	0.076

```
kable(deltas %>% mutate(across(where(is.numeric), round, 3)),  
      caption = "Delta fit (CFI, RMSEA) thresholds.")
```

Table 9: Delta fit (CFI, RMSEA) thresholds.

step	dCFI	dRMSEA
Configural -> Metric	-0.001	-0.001
Metric -> Scalar	0.005	-0.008

step	dCFI	dRMSEA
------	------	--------

9 Reproducibility appendix

```
sessionInfo()
```

```
## R version 4.6.1 (2026-06-24)
## Platform: x86_64-pc-linux-gnu
## Running under: Ubuntu 24.04.5 LTS
##
## Matrix products: default
## BLAS: /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.0
## LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.0 LAPACK version 3.12.0
##
## locale:
##  [1] LC_CTYPE=en_US.UTF-8      LC_NUMERIC=C              LC_TIME=cs_CZ.UTF-8      LC_COLLATE=en_US.UTF-8
##  [5] LC_MONETARY=cs_CZ.UTF-8   LC_MESSAGES=en_US.UTF-8   LC_PAPER=cs_CZ.UTF-8     LC_NAME=C
##  [9] LC_ADDRESS=C             LC_TELEPHONE=C            LC_MEASUREMENT=cs_CZ.UTF-8 LC_IDENTIFICATION=C
##
## time zone: Europe/Prague
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats4      stats      graphics  grDevices  utils      datasets  methods    base
##
## other attached packages:
## [1] mirt_1.47      lattice_0.23-1 car_3.1-5      carData_3.0-6 semTools_0.5-9 lavaan_0.7-2 difR_6.1.0
## [8] janitor_2.2.1 stringr_1.6.0 knitr_1.51     psych_2.6.5    ggplot2_4.0.3 tidyr_1.3.2    dplyr_1.2.1
##
## loaded via a namespace (and not attached):
##  [1] RColorBrewer_1.1-3  rstudioapi_0.19.0  audio_0.1-12      shape_1.4.6.1      magrittr_2.0.5
##  [6] farver_2.1.2        nloptr_2.2.1       rmarkdown_2.32    fs_2.1.0           vctr_0.7.3
## [11] splines2_0.5.4      minqa_1.2.8        tinytex_0.61      htmltools_0.5.9    forcats_1.0.1
## [16] haven_2.5.5         cellranger_1.1.0   Formula_1.2-6     dcurver_0.9.3      parallelly_1.48.0
## [21] testthat_3.3.2      rootSolve_1.8.2.4  lubridate_1.9.5   admisc_0.41        lifecycle_1.0.5
## [26] iterators_1.0.14    pkgconfig_2.0.3    Matrix_1.7-6      R6_2.6.1           fastmap_1.2.0
## [31] rbibutils_2.4.1     future_1.75.0      snakecase_0.11.1  digest_0.6.39      Exact_3.3
## [36] vegan_2.7-6         progressr_1.0.0    timechange_0.4.0  httr_1.4.9         abind_1.4-8
## [41] mgcv_1.9-4          compiler_4.6.1     proxy_0.4-29      withr_3.0.3        S7_0.2.2
## [46] R.utils_2.13.0      MASS_7.3-66        sessioninfo_1.2.4 GPArotation_2026.8-2 permute_0.9-10
## [51] gld_2.6.8           tools_4.6.1        pbivnorm_0.6.0    otel_0.2.0         future.apply_1.20.2
```

## [56]	clipr_0.8.1	R.oo_1.27.1	glue_1.8.1	quadprog_1.5-8	nlme_3.1-171
## [61]	grid_4.6.1	cluster_2.1.8.2	generics_0.1.4	gtable_0.3.6	tzdb_0.5.0
## [66]	R.methodsS3_1.8.2	class_7.3-24	data.table_1.18.6.1	lmom_3.3	hms_1.1.4
## [71]	stringfish_0.19.2	Deriv_4.3.5	foreach_1.5.2	pillar_1.11.1	splines_4.6.1
## [76]	survival_3.8-12	tidyselect_1.2.1	pbapply_1.7-5	reformulas_0.4.4	gridExtra_2.3.1
## [81]	deltaPlotR_1.9	xfun_0.60	expm_1.0-1	brio_1.1.5	stringi_1.8.7
## [86]	VGAM_1.1-14	yaml_2.3.12	boot_1.3-32	evaluate_1.0.5	codetools_0.2-20
## [91]	beepR_2.0	msm_1.8.2	tibble_3.3.1	cli_3.6.6	RcppParallel_6.2.1
## [96]	DescTools_0.99.60	Rdpack_2.6.6	Rcpp_1.1.2	readxl_1.5.0.1	globals_0.19.1
## [101]	polycor_0.8-2	parallel_4.6.1	readr_2.2.0	lme4_2.0-6	listenv_1.0.0
## [106]	glmnet_5.1	mvtnorm_1.4-2	SimDesign_2.27	scales_1.4.0	e1071_1.7-17
## [111]	purrr_1.2.2	rlang_1.3.0	qs2_0.3.1	mnormt_2.1.2	ltm_1.2-0